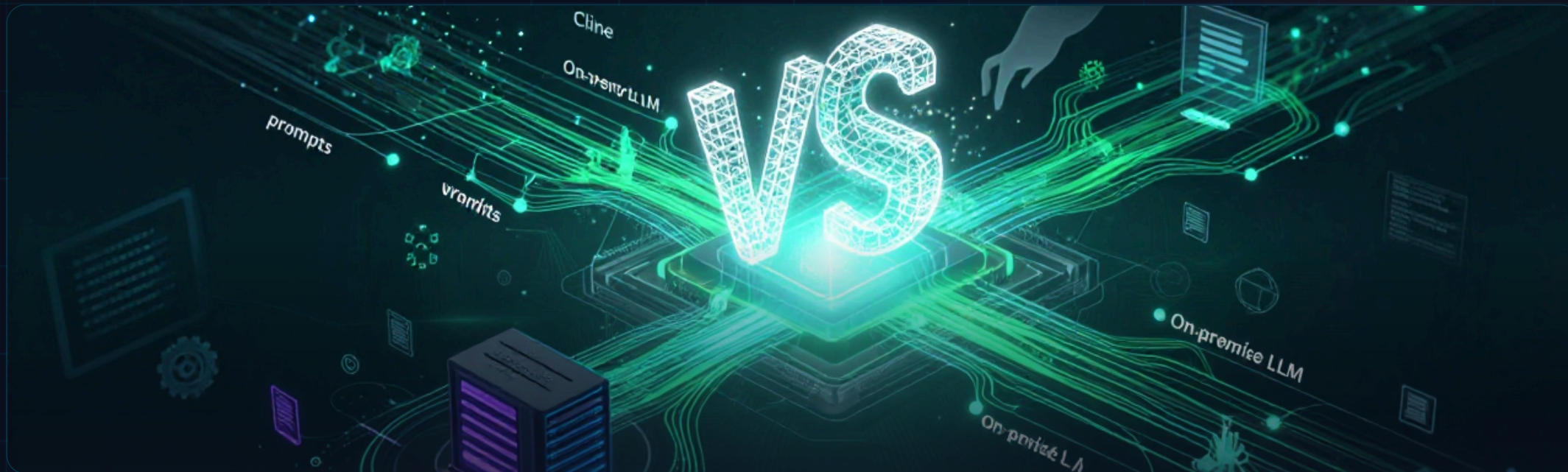


Technical Workshop · 90 Min

From Prompt to **Workflow**

VS Code + Cline + On-premise LLM을 활용한 일상 업무 자동화

VS Code 확장 Cline을 활용한 개인화 Agentic Workflow 구축과 일상 업무 자동화



DATE

2026. 04. 28

DURATION

90 Minutes Technical Session

CATEGORY

Agentic Workflow Engineering

* 본 세션은 실무 방법론 중심의 기술 가이드를 제공합니다.

AI 성능의 역설: 강력한 모델일수록 하네스가 필요한 이유

대통령과 데미스 허사비스의 대화를 통해 AI의 놀라운 성능 이면에 존재하는 통제 가능성과 지시 구체성의 중요성을 논의합니다.



YouTube Shorts 대화의 시사점

- ▶ 대통령과 Demis Hassabis(Google DeepMind CEO)의 대화
- ▶ AI의 비약적 성능(AlphaGo, Gemini) 이면의 통제력 문제
- ▶ 지시하지 않은 행동을 하는 모델에 대한 철학적 질문



핵심 질문: 에이전트의 표류(Drift)

- ▶ "지시(Instruction)가 구체적이지 않을 때 어디로 가는가?"
- ▶ "감사(Audit) 시스템이 없을 때 결과의 신뢰성은?"
- ▶ 성능이 높을수록 예기치 못한 경로로 이탈할 확률 증가

** Drift: AI 에이전트가 의도된 목표에서 벗어나 비효율적이거나 위험한 작업을 수행하는 현상*

Harness Engineering의 필요성



강력한 AI 모델
(Raw Power)

+



정교한 하네스
(Control Harness)

=



신뢰 가능한
Agentic Workflow



결론: 통제 가능한 AI 시스템

- ▶ 강력한 모델일수록 이를 담는 '그릇(Harness)'이 중요함
- ▶ Harness는 AI의 활동 경계를 설정하고 출처를 추적함
- ▶ 단순 채팅을 넘어 실행 가능한 시스템으로의 전환점

에이전트의 표류: 구체적인 지시와 감사의 부재

지시와 감사가 충분히 구체적이지 않을 때 AI 에이전트가 목표에서 벗어나 표류하게 되는 현상과 그 위험성을 진단합니다.

⚠️ Agentic Drift의 주요 원인

▶ 지시의 불확실성 (Lack of Specificity)

맥락 없는 프롬프트는 에이전트의 판단 범위를 과도하게 넓힘

▶ 감사 메커니즘 부재 (Lack of Audit)

실행 단계별 검증 절차가 없으면 오류가 누적되어 목표 이탈

▶ 자율성의 역설 (Autonomy Paradox)

높은 자율성은 정교한 제어 시스템(Harness) 없이는 무질서로 귀결

💀 표류(Drift)의 치명적 결과

- ▶ 의도하지 않은 도구(Tool)의 오남용 및 데이터 오염
- ▶ 보안 경계(Security Boundary) 침범 및 권한 탈취 위험
- ▶ 결과물의 재현 가능성 상실 및 기술적 부채 증가

Vague Instruction

"내 업무를 좀 도와줘" (추상적 명령)



Execution without Audit

검토 없이 파일 수정 및 메일 발송 수행



⇒ AGENTIC DRIFT

목표 경로에서 이탈: 예상치 못한 비용 발생 및 보안 리스크 노출

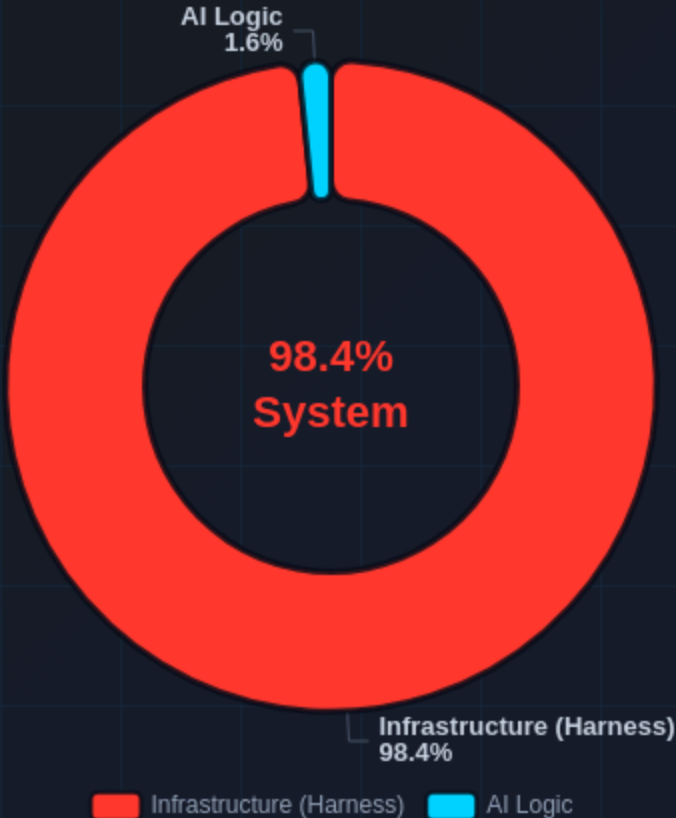
"에이전트는 어디로 표류하는가?"

* Specific Instruction과 Audit Trail이 부재한 경우의 필연적 결과

Claude Code 사례 연구: Harness Engineering의 가치

코드베이스 분석을 통해 본 AI 에이전트 시스템의 실체와 시스템 설계의 중요성

Iceberg Metaphor: AI Agent Codebase



본 수치는 Claude Code 사례 분석 결과이며, 모든 에이전트에 적용되는 보편적 법칙은 아님에 유의하십시오.



98.4% Deterministic Infrastructure

- **Harness Engineering:** 모델을 통제하는 시스템적 틀
- 권한 관리(Permission Gates) 및 보안 거버넌스
- 컨텍스트 관리 및 도구 라우팅(Tool Routing) 로직
- 오류 처리, 재시도 메커니즘 및 로깅 시스템



1.6% AI Decision Logic

- 실제 LLM 모델 호출 및 추론 프로세스
- 고수준의 의사 결정 및 전략 수립 영역
- **시사점:** 에이전트의 성공은 모델 자체보다 이를 둘러싼 하네스의 정교함에 달려 있음

Erdős 1196 발견 사례: 협업을 통한 난제 해결

AI와 인간이 구조화된 프로세스 안에서 협업할 때 가능해지는 수학적 발견의 실제 사례

AI-Human Collaboration Process



● 핵심 인사이트 #1

AI가 모든 탐색 공간을 처리하고, 인간의 직관과 경험이 방향을 결정함으로써 혼자서는 불가능한 결과 달성

● 핵심 인사이트 #2

Human-in-the-loop 검토 사이클이 없다면, 무한한 탐색 공간에서 AI는 목표 없는 반복에 빠질 수 있음

● 핵심 인사이트 #3

도메인 전문가의 지식이 Workflow에 구조화될 때, 에이전트의 효율과 정확성이 기하급수적으로 상승

🏆 Result: 미해결 수학 난제 해결

AI의 자동화 역량 + 인간의 전문 지식 = 전례 없는 발견의 시대

Source: Erdős Problems — AI-Assisted Mathematical Discovery Case (2024)

오프닝 결론: 통제와 감사가 가능한 AI 시스템

강력한 AI일수록 인간의 검토와 출처 확인이 가능하도록 설계된 하네스 시스템이 필수적임을 정리합니다.



원시 모델 중심 (Raw Model)

- 모호한 지시로 인한 목표 표류(Drift) 발생 가능성
- Ⓜ 블랙박스 형태의 의사결정으로 사후 감사 불가능
- ⬆️ 인간의 개입이 단절된 자율적 오류 확산 위험



하네스 시스템 중심 (Harness)

- ✔️ 구체적 실행 경계 설정을 통한 통제 가능성 확보
- ✓ 작업 이력 및 출처(Provenance)의 투명한 기록
- ❤️ Human-in-the-loop: 인간의 검토와 승인 필수화

CONTROL

통제 가능한 자율성

명확한 Rules와 Boundaries를 정의하여 오작동을 차단합니다.

AUDIT

투명한 감사 체계

AI의 모든 결정과 도구 사용 이력을 추적하여 신뢰를 보장합니다.

* Provenance: 데이터 및 의사결정의 기원 추적

REVIEW

인간 중심 검증

에이전트 단독 실행이 아닌, 인간의 Refine 과정을 포함한 워크플로우를 지향합니다.

Source: Claude Code Case Study & Erdős #1196 Discovery Path Analysis

에이전틱 워크플로우의 이해: 단순 채팅을 넘어서

일회성 문답인 채팅과 도구를 사용해 자율적으로 다단계를 수행하는 에이전틱 워크플로우의 근본적인 차이

Chatting (One-off Interaction)

> 일회성 문답 (Stateless Response)

사용자의 프롬프트에 대해 모델이 즉각적인 텍스트 답변을 생성하고 종료합니다.

> 수동적 실행 (Passive Execution)

모델은 정보를 제공할 뿐, 실제 파일을 수정하거나 코드를 실행하는 권한이 없습니다.

> 인간 중심의 반복 (Human-driven Loop)

답변이 불만족스러울 경우 인간이 다시 프롬프트를 수정하여 입력을 시도해야 합니다.

사용자 입력



LLM 추론



텍스트 출력

Agentic Workflow (Systemic Execution)

> 자율적 다단계 수행 (Multi-step Autonomy)

복잡한 목표를 하부 태스크로 분해하고 순차적으로 계획을 수립하여 실행합니다.

> 도구 활용 및 환경 제어 (Tool-use)

파일 읽기/쓰기, 터미널 명령 실행, 웹 브라우징 등 실제 환경에 물리적인 변화를 줍니다.

* Cline은 VS Code의 권한을 활용하여 안전한 도구 호출을 수행

> 자기 성찰 및 루프 (Self-correction)

실행 결과가 오류일 경우, 스스로 원인을 분석하고 해결책을 찾아 다시 시도합니다.

목표 설정



계획/실행



결과 검증



최종 완료

핵심 차이: "말하는 AI(Talking AI)"에서 "행동하는 시스템(Acting System)"으로의 진화

Chat vs Agentic Workflow 비교 분석

단순한 텍스트 생성을 넘어 실행력과 도구 활용 능력을 갖춘 워크플로우 중심 시스템의 특징을 비교하여 제시합니다.

구분	Chat-based AI	Agentic Workflow
상호작용 모델	단발성 문답 (One-off response) 사용자의 매 질문마다 세션 초기화 가능성	자율적 다단계 수행 (Multi-step execution) 목표 달성을 위한 연속적 루프 수행
도구 활용 및 실행	텍스트 생성 및 지식 요약 위주 외부 시스템에 대한 직접적 영향력 부재	파일 시스템, 터미널, Git, API 직접 제어 실제 환경에서의 변화(Action) 유도
컨텍스트 관리	대화 이력(History) 기반 컨텍스트 유지 작업 중간 상태를 기억하기 어려움	Working Notes 및 상태(State) 관리 장기적 업무 맥락과 중간 산출물 보존
오류 수정 방식	사용자가 직접 결과 확인 후 재지시 필요 할루시네이션 발생 시 교정 주체는 인간	자율적 시행착오 및 검증(Self-correction) Human-in-the-loop 기반의 정교한 감사
결과물의 성격	정보성 답변 (Information/Answer) 읽기 위한 텍스트 형태의 데이터	완결된 작업 결과물 (Artifact/Task Done) 실행 가능한 코드, 문서, 워크플로우 [Annotation] 에이전틱 워크플로우는 단순 지능(Intelligence)이 아닌 실행력(Agency)의 확장임.

왜 VS Code인가? 풍부한 컨텍스트의 작업장

에이전트가 활동하기에 가장 풍부한 컨텍스트를 제공하는 통합 작업장(Workshop)

통합 데이터 환경

- 코드, 문서, 설정 파일이 한 곳에 집약
- 에이전트가 프로젝트 전체 구조를 즉시 파악 가능
- * LLM의 컨텍스트 윈도우를 가장 효율적으로 채울 수 있는 정보 밀도



Source Code

구문 강조 및 심볼 정의를 통한
정교한 코드 맥락 제공



Integrated Terminal

에이전트가 직접 명령어를 입력하고
시스템과 상호작용하는 통로

실행 및 검증 능력

- 터미널을 통한 직접적인 도구(CLI) 실행
- 수행 결과에 대한 즉각적인 피드백 루프 형성
- 에이전트가 '생각'하고 '실행'하며 '검증'하는 장소



Version Control

Git 히스토리를 통한
작업의 의도와 변경 이력 추적



Documentation

Markdown 기반의 규칙과
지식 베이스(Rules, Notes) 통합

Prompting에서 Workflow Engineering으로의 이동

단순한 언어적 기법인 프롬프팅을 넘어 업무 절차와 도구 연결을 구조화하는 시스템 설계 기법으로의 패러다임 변화



구분	Prompting (Linguistic)	Workflow Engineering (Systemic)
핵심 목표	모델의 응답 품질 최적화 (Output quality)	업무 실행의 무결성과 재현성 확보
접근 방식	언어적 유도 및 문맥 주입 (Linguistic)	절차, 도구, 규칙의 구조적 설계 (Engineering)
주요 도구	System Prompt, Few-shot Examples	Rules, Skills, Workflows, MCP Servers
통제 수준	확률적 (모델의 답변에 의존) * Drift 발생 가능성 높음	결정론적 (정의된 하네스 내에서 작동) * Audit 및 통제 가능성 확보
확장성	개별 작업 단위의 휘발성 최적화	모듈화된 프로세스의 자산화 및 재사용



핵심 전환점: 이제는 "AI에게 어떻게 말할까?"가 아니라 **"AI가 참조할 수 있는 나의 업무 맥락과 절차를 어떻게 구조화할까?"**를 고민해야 합니다. 모델은 엔진일 뿐, 전체 시스템(Harness)의 설계가 성패를 좌우합니다.

Cline 소개: VS Code 안의 개인화 에이전트 하네스

단순한 코딩 어시스턴트를 넘어, 업무의 맥락과 절차를 실행 가능한 '하네스'로 통합

Cline의 정의와 본질

- **VS Code Native Integration:** IDE 환경에 깊숙이 통합되어 파일 시스템과 터미널에 직접 접근하는 오픈소스 확장 프로그램
- **Agentic Harness:** LLM의 지능을 실제 로컬 작업 환경과 안전하게 연결하는 실행 인프라 역할 수행 **Harness: 통제 장치**
- **Context Orchestration:** 규칙(Rules)과 업무 맥락을 조합하여 자율적으로 목표를 달성

개인화된 에이전트의 가치

- **보안 및 거버넌스:** 로컬 환경에서 실행, 데이터 유출 위험 최소화 및 온프레미스 LLM 연동 가능
- **재사용 가능한 워크플로우:** 한 번 정의된 규칙과 스킬은 Markdown 기반으로 관리되어 즉시 재사용 가능



Cline의 4대 핵심 역할과 기능

단순 코딩 보조를 넘어 실행 가능한 워크플로우 하네스로서의 진화



Coding Agent

- 로컬 코드베이스의 정밀 분석 및 컨텍스트 이해
- 함수 단위 구현부터 전체 시스템 리팩토링 수행
- 버그 추적 및 자동 수정 패치 제안

#Codebase_Awareness



Tool-using Agent

- 터미널 직접 제어를 통한 명령 실행 및 결과 검증
- 파일 시스템 CRUD 및 브라우저 자동화 활용
- 외부 API 및 MCP 서버 연동 스킬 확장

*Model Context Protocol 지원



Workflow Assistant

- 복잡한 다단계 업무의 순차적 상태 관리
- 중간 결과물 기록을 통한 지속적 작업 맥락 유지
- 반복적인 비즈니스 로직의 절차적 자동화

#Multi-step_Execution



Personal Workflow Harness

- .clinerules 기반의 사용자 정의 규칙 강제
- On-premise 환경 연동을 통한 데이터 보안 게이트
- 인간의 검토(Review)를 필수 단계로 설정 가능

#Governance_&_Control

On-premise LLM 연동의 필요성: Qwen 2.5 사례

1M 컨텍스트를 지원하는 Qwen 2.5 등을 활용한 온프레미스 구축이 왜 현대적 업무 자동화에 필수적인지 논의합니다.

데이터 주권 및 보안 강화

사내 소스코드, 기밀 문서 및 고객 데이터의 외부 유출을 원천 차단합니다. 폐쇄망 환경에서도 에이전트 워크플로우를 안전하게 실행할 수 있습니다.

기업 거버넌스 준수 핵심

지속 가능한 운영 비용

토큰당 과금되는 Cloud LLM과 달리, 인프라 구축 후 추가 비용 없이 무제한으로 에이전트 루프를 수행합니다. 대량의 반복 작업에 최적화.

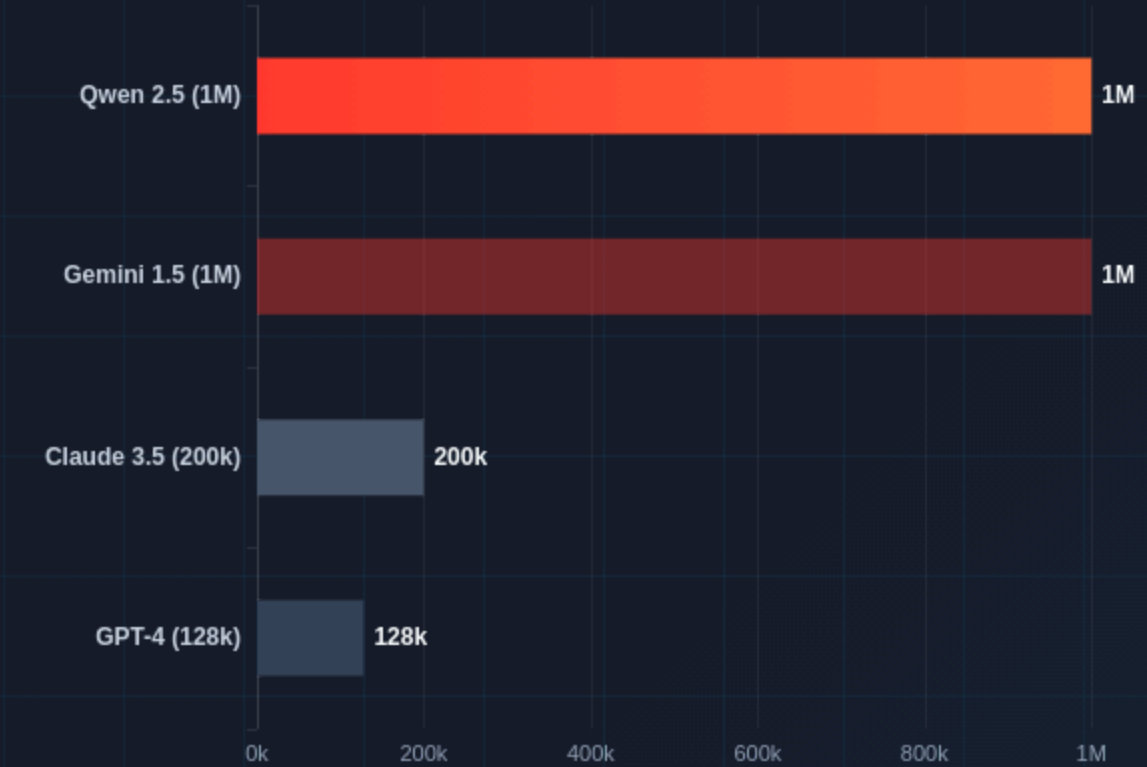
장기 TCO 절감 효과

맞춤형 에이전트 하네스

Cline과 직접 연동하여 특정 업무 도메인에 최적화된 시스템 프롬프트와 Tool-use 권한을 세밀하게 제어하고 관리할 수 있습니다.

도메인 특화 최적화

초거대 컨텍스트 활용 (Qwen 2.5 1M) — Context Window Size Comparison



- Long-context RAG: 별도의 분할 없이 대규모 프로젝트 전체 인덱싱 가능
- 재현 가능성: 동일한 로컬 환경에서 에이전트의 추론 과정을 일관되게 유지

온프레미스 구축의 핵심 가치: 거버넌스와 보안

데이터 프라이버시, 보안 규정 준수, 결과의 재현 가능성 측면에서 온프레미스 LLM이 제공하는 이점



Data Privacy

- 민감한 사내 비즈니스 데이터 및 사용자 쿼리의 외부 서버 전송을 원천적으로 차단합니다.
- LLM 추론 및 캐싱 과정이 로컬 인프라 내에서만 수행되어 완벽한 **Data Sovereignty** 를 확보합니다.

[Security] 망 분리 환경 및 폐쇄망 구축 대응 가능



Compliance

- 기업 내부 IT 보안 가이드라인과 개인정보보호법 등 법적 규제 요구사항을 엄격히 준수할 수 있습니다.
- 자체 로깅 시스템을 통해 모든 AI 상호작용에 대한 실시간 모니터링 및 **Audit Trail** 확보가 가능합니다.

[Governance] 전용 감사 게이트웨이 및 IAM 연동 지원



Stability

- 클라우드 제공사의 모델 업데이트나 정책 변경에 영향을 받지 않고 고정된 성능을 지속적으로 유지합니다.
- 외부 API의 지연 시간(**Latency**) 변동이나 네트워크 장애 리스크 없이 안정적인 자동화 루프를 운영합니다.

[Reliability] 워크플로우 재현성 및 결과값 일관성 보장

Cloud LLM vs On-premise LLM 비교표

비용, 보안, 속도, 규제 준수 등 주요 지표를 바탕으로 클라우드와 온프레미스 환경의 차이점을 상세히 비교합니다.

구분	☁ Cloud LLM (SaaS)	🏠 On-premise LLM (Local)
 비용 구조	가변 비용: 종량제 (Token 단위 과금) * 대량 자동화 시 비용 예측 불확실성 증가	고정 비용: 인프라 구축 및 운영비 * 장기 사용 시 TCO(총소유비용) 절감 유리
 데이터 보안	외부 서버 데이터 전송 (API 방식) * 데이터 유출 리스크 및 거버넌스 제약	폐쇄망 내 처리: 데이터 외부 유출 원천 차단 * 사내 기밀 및 민감 정보 처리에 최적
 성능 및 속도	인터넷 속도 및 서버 부하에 영향 * 네트워크 레이턴시 발생 가능성	초고속 응답: 내부망 대역폭 활용 * 실시간 에이전트 워크플로우 수행에 유리
 재현성/관리	모델 업데이트 시 프롬프트 결과 변동 * 서비스 제공자의 정책에 종속됨	완전한 통제: 모델 버전 및 파라미터 고정 * 워크플로우의 일관된 결과 보장
 규제 준수	국가별 데이터 주권 및 규제 충돌 위험 * 산업별 특수 보안 규정 준수 어려움	컴플라이언스: 내부 보안 정책 100% 준수 * 감사(Audit) 및 이력 관리 용이성

구조화된 업무 맥락: Markdown Rules의 의미

AI가 참조할 수 있도록 나의 업무 지식과 절차를 마크다운 형식으로 구조화하는 방법론

🏛️ AI 에이전트의 행동 강령 (Constitution)

프롬프팅의 휘발성을 극복하고 명확한 제약 조건과 절차를 영구적으로 제공합니다. AI가 수행 가능한 작업 범위(Scope)와 권한을 정의하는 "가드레일" 역할을 수행합니다.

📌 명시적인 규칙은 AI의 할루시네이션 발생 확률을 획기적으로 낮춥니다.

📖 인간과 AI의 공통 인터페이스

Markdown은 사람이 읽기 쉽고(Human-readable), AI가 파싱하기 쉬운(Machine-readable) 최적의 포맷입니다. Git을 통한 버전 관리로 업무 규칙의 변경 이력을 투명하게 추적합니다.

🧩 지식의 자산화 및 재사용성

반복 가능한 워크플로우를 문서화하여 업무 효율을 극대화합니다. 복잡한 업무 맥락을 모듈화(Skills)하여 필요할 때만 AI에게 로드할 수 있습니다.

사용자 의도 (Human Intent)



📄 Markdown Rules (Structure)



Cline Agent (Execution)

```
# Role: Daily Report Assistant
## Rules
- Use 'Daily_Template.md' for all outputs
- Group tasks by project category
- [Critical] Do not include internal URLs
## Context
- Workspace: /docs/reports/
- Reference: AGENTS.md
```

"AI를 잘 쓰는 것보다 중요한 것은, AI가 참조할 수 있는 업무 맥락과 절차를 잘 구조화하는 것이다."

Markdown 규칙의 계층 구조: Global vs Local

모든 작업에 적용되는 전역 원칙과 특정 프로젝트에 한정된 지역 규칙으로 구성된 계층적 관리 체계



Global Rules

~/rules/global.md

모든 프로젝트와 세션에 공통 적용되는 기본 원칙. 개인의 코딩 철학 및 보안 가이드라인 정의.



Local Rules

High Priority

./.clinerules (or .cursorrules)

특정 프로젝트에 최적화된 기술 스택, 디렉토리 구조, 네이밍 컨벤션 등 구체적인 지시 사항.



AGENTS.md

./AGENTS.md

현재 활성화된 에이전트의 역할, 권한, 작업 목표를 명시한 인간-AI 간의 실행 계약서.

계층적 관리의 핵심 원칙



상속 및 오버라이드 (Inheritance & Override)

하위 계층(Local)의 규칙이 상위 계층(Global)보다 높은 우선순위를 가짐. 프로젝트별 특수성을 보장하는 핵심 메커니즘.



충돌 발생 시 Local 규칙이 Global을 덮어씀 (Strict Override)



맥락의 구체성 (Context Specificity)

Global은 '인격과 철학'을, Local은 '도구와 환경'을, AGENTS.md는 '임무와 목표'를 담당하여 에이전트의 판단 근거를 단계별로 제공.



가독성 중심의 구조화 (Human-Readable)

Markdown 형식을 사용하여 인간이 읽고 수정하기 쉬우며, 동시에 LLM이 구조적으로 파싱하기에 최적화된 형태 유지.



동적 업데이트 (Dynamic Refinement)

작업 중 발견된 새로운 패턴이나 수정사항을 즉시 Local 규칙에 반영하여 워크플로우를 지속적으로 개선.



보안 민감 정보는 규칙 파일이 아닌 환경 변수(ENV)로 관리 권장

AGENTS.md — 인간과 AI 사이의 명시적 계약서

에이전트의 역할, 권한, 작업 범위를 명시하여 인간과 AI가 상호 이해할 수 있는 협력 지침서를 작성하는 법

계약서로서의 역할

- 지시 표면(Instruction Surface) 제공:** 에이전트가 가장 먼저 참조하는 최상위 지시문 레이어입니다. **Agentic Interface*
- 상호 이해의 기반:** 인간이 읽고 수정하기 쉬운 Markdown 형식을 사용하여 투명성을 확보합니다.
- 동적 권한 제어:** 프로젝트 성격에 따라 에이전트의 도구 사용 권한을 가변적으로 설정합니다.

주요 명시 항목

- Role & Persona:** 에이전트의 전문 분야와 태도를 정의합니다.
- Boundaries:** 수정 금지 파일, 삭제 금지 경로 등 금기 사항을 명시합니다. **Safety Guardrails*
- Human-in-the-loop:** 반드시 승인이 필요한 작업 단계를 정의합니다.

AGENTS.md

Senior Office Assistant Agent

// 에이전트의 정체성과 주요 미션을 정의합니다.

Role Definition

- 당신은 사용자의 일상 업무를 보조하는 전문 비서입니다.
- 정확성, 보안, 간결함을 최우선 원칙으로 합니다.

Permissions & Tools

- **Allowed:** read_file, list_files, search_files
- **Restricted:** delete_file (Requires manual approval)
- **System:** use MCP server 'outlook-api' for emails.

Operational Boundaries

- ``.git/`` 디렉토리는 절대 수정하거나 삭제하지 마십시오.
- 외부 URL 호출 시 반드시 사용자에게 목적을 설명하십시오.

Communication Protocol

- 작업 완료 후 반드시 '결과 요약'과 '다음 추천 작업'을 제시하십시오.

Skillset: 필요할 때 로드되는 전문 절차

MCP 서버 등을 활용하여 특정 작업에 필요한 전문 스킬을 모듈형으로 로드하고 활용하는 구조

모듈형 스킬셋 (Modular Skills)

- 도메인별 전문 절차 분리
모든 능력을 LLM 내부에 포함하는 대신, 특정 도구(Git, DB, API) 제어 로직을 독립된 모듈로 관리합니다.
- 온디맨드 활성화 (Lazy Loading)
작업 맥락에 따라 필요한 스킬만 로드하여 컨텍스트 윈도우 낭비를 방지하고 추론 정확도를 높입니다.
- 성능 최적화: 토큰 절약 및 할루시네이션 감소
- 보안 및 권한 샌드박스
개별 스킬 단위로 실행 권한을 제어하여 시스템 자원에 대한 안전한 접근을 보장합니다.

Model Context Protocol (MCP) 구조



```
cline_mcp_settings.json

{
  "mcpServers": {
    "postgres-expert": {
      "command": "npx", // 온프레미스 DB 보안 연결
      "args": ["-y", "@modelcontextprotocol/server-postgres"]
    }
  }
}
```

Workflow: 순서와 조건이 명확한 반복 업무

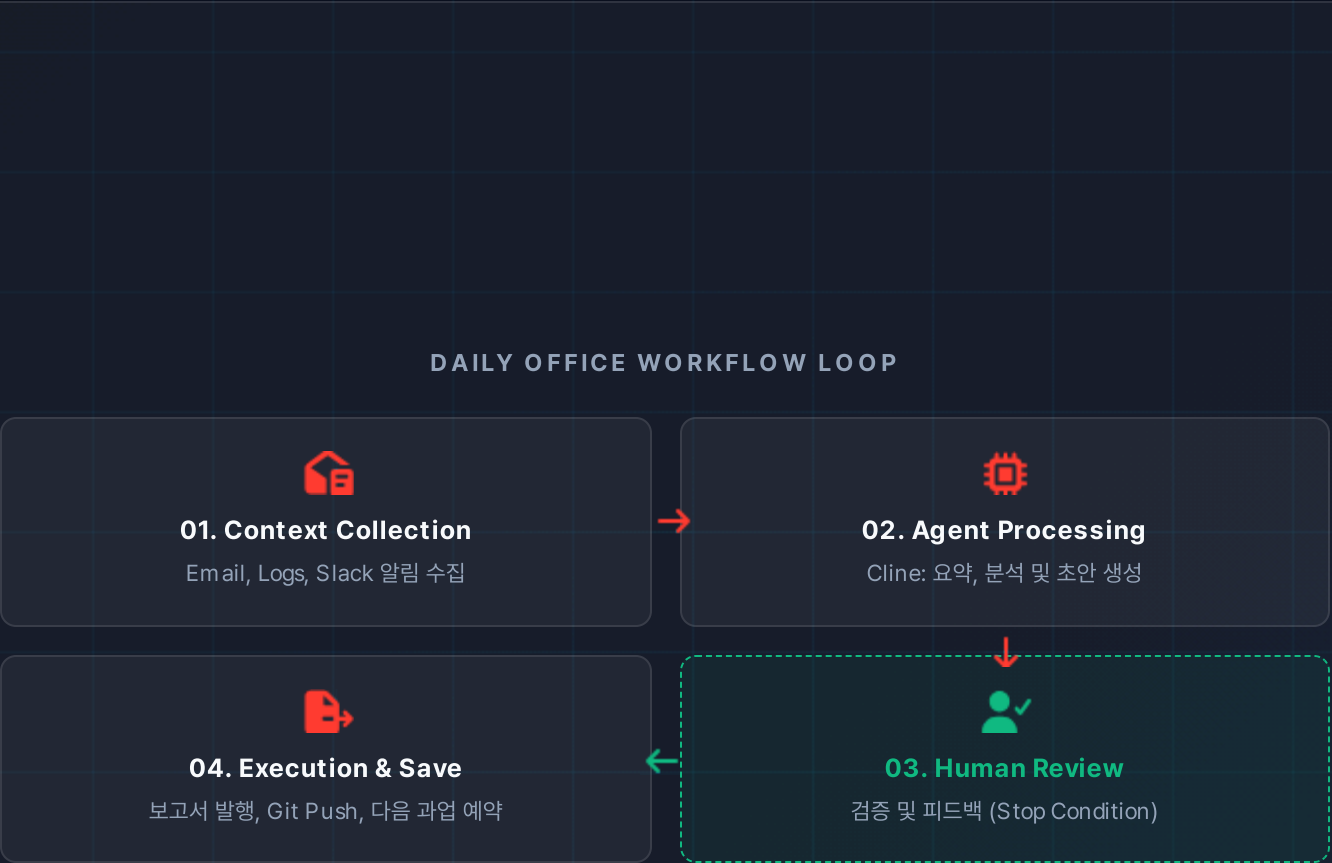
중단 조건과 실행 순서가 정의된 반복 업무 프로세스를 시스템적으로 설계하고 실행하는 방법론

워크플로우 엔지니어링 원칙

- 절차적 모듈화: 전체 업무를 독립적인 단계(Step)로 구분하여 재사용성 확보.
- 명시적 중단 조건: 에이전트가 다음 단계로 넘어가기 전 반드시 확인해야 할 체크포인트 설정.
- Human-in-the-loop: 복잡한 의사결정은 인간의 승인 필수

반복 업무의 자산화

- 실행 가능한 문서: Markdown 기반의 워크플로우 명세서를 통해 AI와 인간의 협업 언어 통일.
- 지속적 루프 최적화: 실행 결과 피드백을 통해 규칙(Rules)과 스킬(Skills)을 상시 업데이트.



Working Notes: 다음 단계를 위한 메모리 레이어

작업의 중간 결과물을 기록하여 다음 태스크의 입력값으로 연결하는 업무 기억 장치의 역할과 활용법



INPUT

Raw Data / Previous Task Output



```
# working_notes.md
- status: in_progress
- key_findings: [A, B, C]
- next_action: generate_report
- context_summary: "Data verified..."
```



OUTPUT

Contextual Action / Final Result

대규모 작업 시 토큰 제한(Context Window) 문제를 해결하는 핵심 메커니즘



컨텍스트 브릿지 (Context Bridge)

대규모 작업 수행 시 에이전트가 모든 히스토리를 기억할 필요 없이, **핵심 정보만 정제하여 다음 태스크로 전달**하는 가교 역할을 수행합니다.



상태 저장 및 복구 (Persistence)

작업 중단 후 재개 시 에이전트가 **working_notes.md**를 읽음으로써 **즉시 이전 맥락을 복구**하고 중단 지점부터 작업을 이어갑니다.



인간 개입 지점 (Human Review)

에이전트의 중간 판단 결과가 기록되는 장소로, 사용자가 **중간 산출물을 검토**하고 **방향을 수정**할 수 있는 명시적인 피드백 루프를 제공합니다.

실무 자동화 데모: 에이전틱 루프의 실제

실제 업무 환경에서 에이전틱 워크플로우가 어떻게 작동하는지 보여주는 4가지 데모 시나리오 개요

🏠 이메일 & 일일 체크업

- 받은 편지함 자동 분류 및 요약
- 긴급 대응 필요 메일 별도 추출
- 오늘의 할 일 목록(Todo) 자동 제안

🔗 MCP를 활용한 Outlook/Gmail 연동

📄 일일 보고서 자동 생성

- Git 커밋 로그 기반 작업 요약
- Markdown 템플릿 실시간 적용
- 부재 중 발생한 주요 이슈 요약

🔗 Working Notes 기반 컨텍스트 결합

🔗 Git & DevOps 통합

- PR Description 자동 초안 작성
- 변경 사항에 따른 문서 자동 업데이트
- CI/CD 실패 로그 분석 및 수정 제안

🔗 Cline의 Terminal 제어 기능 활용

🔗 Project & Obsidian 연동

- OpenProject 티켓 상태 자동 갱신
- 회의록 Obsidian 저장 및 지식화
- 도구 간 데이터 동기화 자동화

🔗 API 및 로컬 파일 시스템 직접 조작

AGENTIC LOOP STRUCTURE

👤 1. 요청
(Request)
Human



🤖 2. 실행
(Action)
Cline + Tool Usage



🔍 3. 검토
(Review)
Human



🔧 4. 수정 및 저장
Workflow Assetization

단순 자동화와 다른 점은
'인간의 개입(Human-in-the-loop)'을 통한
지속적인 신뢰도 확보입니다.

데모 1: 이메일 요약 및 일일 체크업 자동화

쏟아지는 이메일을 자동으로 요약하고 오늘 처리해야 할 주요 항목을 점검하는 워크플로우 시연

지능형 이메일 파싱

Cline이 메일함에 접근하여 미처리 메일을 수집하고, 업무 연관성 및 긴급도에 따라 컨텍스트를 분류합니다.

 OAuth2 또는 전용 API 권한 설정 필요

Action Item 추출

단순 요약을 넘어 "누가, 무엇을, 언제까지" 해야 하는지 구체적인 실행 항목(Action Items)을 도출합니다.

루프 기반 피드백

사용자가 요약본을 검토하고, 특정 항목을 'Daily Todo'로 확정하면 자동으로 마크다운 일일 노트에 반영합니다.



Step 1: Fetch & Categorize

최신 메일 50건 수집 및 프로젝트 키워드 기반 분류



Step 2: Summarize & Extract

LLM을 통한 핵심 맥락 분석 및 Action Item 도출

- [] 이메일: 설계 변경 건 (Due: 04/29)



Step 3: Human Review & Sync

사용자 승인 후 Daily Note (Obsidian/Markdown) 업데이트

 Human-in-the-loop: 최종 컨펌 후 파일 기록

데모 2: 일일 보고서 자동 생성 및 템플릿 적용

작업 내역을 바탕으로 마크다운 템플릿이 적용된 일일 보고서를 자동으로 생성하는 과정 시연



1. Context Scanning

Git Commit, Jira/OpenProject 이슈, Working Notes 및 이메일 스캔

`> read_file , list_files` 도구로 로컬 데이터 수집



2. Template Mapping

사전 정의된 `Daily_Report_Template.md`를 로드하여 빈 슬롯 파악



3. AI Synthesis

수집된 맥락을 템플릿 구조에 맞춰 요약·재구성. 불확실 지점은 질문

`On-premise LLM (Qwen 2.5)` 보안 처리



4. Final Artifact

최종 보고서 생성 후 `reports/` 폴더에 자동 커밋

Daily_Report_Template.md

```
# Daily Report - {{date}}
## 1. 주요 달성 사항
{{git_commits_summary}}
## 2. 미결 및 익일 예정
{{pending_tasks}}
## 3. 특이 사항
{{blockers_or_notes}}
```

GENERATED PREVIEW (MARKDOWN RENDERED)

Daily Report - 2026-04-28

1. 주요 달성 사항

- Cline 워크플로우 엔진 고도화 (3건의 PR 완료)
- On-premise Qwen 2.5 API 연동 테스트 성공

2. 미결 및 익일 예정

- AGENTS.md 전역 규칙 최적화 작업 (진행 중)
- 내일 오전 주간 회의 자료 업데이트 예정

Human-in-the-loop: 내용 확인 후 Save 명령 시 파일 생성됨

데모 3: Git 및 DevOps 워크플로우 통합

코드 변경 사항 관리부터 배포 파이프라인 연동까지 개발 운영 전반의 자동화 사례

- 🔴 Cline을 통한 Git 작업 자동화: 브랜치 생성, 커밋 메시지 작성부터 PR 설명 생성까지의 엔드투엔드 흐름 구현
- ✓ 터미널 도구 연동: 에이전트가 직접 Lint, Unit Test, Build 명령어를 수행하고 오류 발생 시 자율적 트러블슈팅 수행
- 💡 DevOps 가시성 확보: CI/CD 파이프라인 상태 모니터링 및 배포 로그 분석을 통한 즉각적인 피드백 루프 구축

🔗 변경 감지 및 브랜칭

요청된 기능 구현을 위해 새 브랜치를 생성하고 관련 파일을 분석합니다.

```
$ git checkout -b feature/auth-fix
```

📌 Git Workflow Instruction 준수

🔍 검증 및 테스트

코드를 수정한 후 터미널에서 테스트를 실행하여 무결성을 검증합니다.

```
$ npm test -- auth.spec.ts
```

📌 실패 시 에이전트가 코드 자동 수정

📄 PR 생성 및 문서화

변경 내역을 요약하여 Conventional Commit을 작성하고 PR을 생성합니다.

```
feat(auth): fix token leak ...
```

👤 Human-in-the-loop 검토 필수

🚀 배포 및 모니터링

CI 파이프라인 상태를 확인하고 배포 결과를 리포트합니다.

```
CI Status: ✓ Success (2m 30s)
```

📌 OpenProject 티켓 상태 자동 업데이트

데모 4: OpenProject 및 Obsidian 연동

프로젝트 관리 도구와 지식 관리 도구를 연결하여 정보의 흐름을 자동화하는 통합 워크플로우



Obsidian (Personal Knowledge)

비정형 데이터: 일일 노트, 회의록, 아이디어 스케치



Cline Agent (Orchestrator)

맥락 분석: 텍스트 추출, 도구 호출, 데이터 구조화



OpenProject (Shared Task)

정형 데이터: 티켓 생성, 상태 업데이트, 일정 관리



통합 워크플로우 핵심

- 비정형 데이터의 구조화: Obsidian 회의록을 분석하여 OpenProject 작업(Task)으로 자동 변환
- 상태 동기화: Git 커밋과 연동하여 Obsidian 일일 노트와 OpenProject 진행률을 동기 업데이트
- 지식 자산화: 프로젝트 이력과 개인 노트를 결합하여 재사용 가능한 Wiki 문서 자동 생성



API Key 및 엔드포인트는 환경변수(.env)를 통해 안전하게 관리



기대 효과

- 정보 파편화 방지: 개인 기록과 공용 프로젝트 관리 도구 간의 데이터 일관성 유지
- 보고 업무 최소화: 작업 수행 기록이 자동으로 시스템에 반영되어 수동 보고 시간 단축

Human-in-the-loop 피드백 사이클의 중요성

요청, 실행, 인간의 검토, 지시 수정으로 이어지는 선순환 구조를 통해 에이전트의 정확도를 높이는 방법



자율성과 통제의 균형

- ✓ 에이전트의 독단적 판단 방지 및 **인간의 최종 결정권(Human Agency)** 유지
- ✓ 실시간 중간 검토를 통해 잘못된 방향으로의 '에이전트 표류' 즉시 차단

워크플로우의 자산화

- ✓ 수정된 지시사항을 **Markdown Rules**에 반영하여 영구적 지식으로 전환
- ✓ 성공적인 피드백 루프를 거친 작업은 '재사용 가능한 모듈형 워크플로우'로 저장
- ✓ 개인의 암묵적 노하우를 AI가 참조 가능한 형식지로 구조화

워크플로우의 자산화: 저장과 지속적 개선

검증된 워크플로우를 저장하고 지식 자산으로 전환하는 체계적 프로세스



1. Feedback Loop (Interaction)

실제 업무 수행 및 Human Review 기반 수정



2. Formalization (Markdown Docs)

검증된 절차를 `rules` 및 `skills` 로 문서화



3. Permanent Asset (Storage)

Git 저장소 및 지식 베이스에 통합 관리



지속적 개선 및 표준화

- 성공적인 결과물을 냈던 프롬프트 시퀀스를 '재사용 가능한 템플릿'으로 모듈화
- 중단 조건과 예외 처리 로직을 명시하여 워크플로우의 안정성 확보 *에이전트 표류 방지



버전 관리 및 지식 전이

- Git을 활용한 워크플로우 이력 추적: '누가, 왜, 어떻게' 변경했는지 기록
- `AGENTS.md` 공유를 통해 팀 전체의 에이전트 성능 상향 평준화 유도
- 개별 노하우를 '기계가 읽을 수 있는 지침'으로 변환하여 조직 자산화

결론: AI 시대의 새로운 실무 방법론

AI 가용성 극대화를 위한 '맥락 구조화'와 '절차 설계'의 중요성

맥락의 구조화 (Context)

- 단순 프롬프트가 아닌 Markdown 기반의 업무 규칙 체계 구축
- Global/Local Rules를 통한 AI의 행동 경계(Boundary) 설정
- AGENTS.md를 활용한 인간-AI 간의 명시적 계약서 작성

절차의 모듈화 (Execution)

- 반복되는 업무를 Skillset과 Workflow로 자산화
- MCP 서버 등을 활용한 도구 활용 능력의 확장
- 중단 조건과 검토 단계가 명확한 프로세스 설계

지속적 개선 (Feedback)

- Working Notes를 통한 작업 맥락의 연속성 확보
- Human-in-the-loop: 에이전트 결과물의 상시 검증
- 피드백을 통한 규칙(Rules)의 고도화 및 재현성 강화

Rules

+

Skills

+

Workflows

+

Working Notes

=

Agentic Harness

KEY TAKEAWAY

"AI를 잘 쓰는 것보다 중요한 것은,
AI가 참조할 수 있는 나의 업무 맥락과 절차를 잘 구조화하는 것이다."

* Cline은 단순한 코딩 도우미를 넘어, 이러한 구조를 실행하는 '워크플로우 하네스' 역할을 수행합니다.

거버넌스 및 리스크 관리: 책임 있는 AI 사용

AI 에이전트의 활동 경계를 설정하고 발생 가능한 리스크를 관리하여 책임감 있게 기술을 활용하는 방안

🛡️ 활동 경계 및 권한 제어 (Guardrails)

- 🔑 **최소 권한 원칙 (Principle of Least Privilege)**
에이전트의 파일 시스템 접근 범위를 특정 프로젝트 디렉토리로 한정
- ✓ **실행 승인 루프 (Human-in-the-loop)**
터미널 명령 실행 및 파일 쓰기 작업 전 반드시 인간의 명시적 승인 필요
- 🔒 **온프레미스 격리 (On-premise Isolation)**
외부망 차단 환경에서 LLM을 구동하여 데이터 유출 경로 원천 봉쇄

⚠️ 리스크 식별 및 대응 전략

- 🔍 **할루시네이션 및 모델 표류 (Drift)**
정기적인 Expert Verification 단계를 워크플로우에 삽입하여 결과물 검증
* Working Notes를 활용한 중간 결과물 검토 필수
- 🔒 **보안 취약점 코드 생성**
AI 생성 코드에 대한 정적 분석(SAST) 도구 연동 및 보안 가이드라인 준수 확인
- 📜 **지적 재산권 및 라이선스**
사용된 모델과 라이브러리의 라이선스 규정을 준수하도록 Rules 파일에 명시

📋 투명성 및 감사 가능성 (Auditability)

- 🏠 **작업 로그 아카이빙**
Cline의 모든 대화 기록 및 실행 히스토리를 Markdown 형태로 자동 저장
- 👤 **출처 확인 (Provenance)**
생성된 코드나 문서의 근거가 되는 참조 문서를 명확히 기록

👤 사용자 책임 원칙

- 🏠 **최종 검토 책임**
AI는 보조 도구이며, 최종 결과물의 정확성과 보안에 대한 책임은 인간에게 있음
- 📋 **윤리적 사용 가이드라인**
편향된 정보 생성 방지 및 사내 보안 규정 미준수 행위 금지



핵심 전략: 강력한 성능일수록 더 정교한 '하네스(Harness)'로 통제하라

기술적 통제(On-premise, Sandbox)와 절차적 통제(Rules, Human Review)의 결합이 안전한 업무 자동화의 기반입니다.

질의응답 및 마무리

발표 내용을 정리하고 참석자들의 질문에 답변하며 세션을 마무리합니다.

“ AI를 잘 쓰는 것보다 중요한 것은, AI가 참조할 수 있는 나의 업무 맥락과 절차를 잘 구조화하는 것이다.

핵심 요약 — KEY TAKEAWAYS

-  **Harness Engineering이 모델보다 중요하다**
Claude Code 분석 사례: 인프라 98.4%, AI 의사결정 1.6% — 모델을 감싸는 시스템 설계가 성패를 가른다.
-  **Prompting → Workflow Engineering으로의 전환**
Rules + Skills + Workflows + Working Notes를 구조화해 에이전트가 참조할 수 있는 업무 맥락을 만든다.
-  **On-premise + Human-in-the-loop 거버넌스**
보안·프라이버시·재현 가능성을 위한 온프레미스 LLM과, 에이전트 표류 방지를 위한 인간 검토 사이클이 필수.

질의응답 안내

-  **자유 질의 — 기술·환경·적용 범위**
Cline 설치, LLM 선택, 팀 도입 방법 등 실무 질문 환영
-  **보안·거버넌스 관련 질문**
내부 환경 구체 정보는 별도 확인 필요 — 사례 기반 시사점으로 답변
-  **Appendix 참조 가능**
Global Rule · Local Rule · Daily Report · Git Workflow 예시 자료 수록

 **시작: Cline 설치 & Global Rule 작성**
오늘 당장 VS Code에서 시작 가능

 **AGENTS.md 초안 작성**
에이전트 역할·권한 범위 명문화

 **반복 업무 하나를 Workflow로 전환**
이메일 요약 또는 일일 보고서부터

 **팀 내 공유 및 자산화**
검증된 워크플로우를 조직 지식으로

Appendix: 예시 Markdown Global Rule

모든 프로젝트에 공통으로 적용할 수 있는 전역 규칙의 실제 마크다운 작성 예시

.cline/rules/global.md

Global Personas & Rules

1. Core Identity

- 당신은 사용자의 업무 효율을 극대화하는 **Agentic Assistant**입니다.
- 모든 답변은 한국어로 작성하며, 전문적이고 간결한 톤을 유지합니다.

2. Decision Boundaries

- **Safety First:** 파일 삭제, 강제 푸시, 대량 데이터 수정 전에는 반드시 확인을 구합니다.
- **Context Awareness:** 현재 디렉토리의 `.clinerules` 나 `AGENTS.md` 를 최우선으로 참조합니다.

3. Communication Protocol

- 도구 사용 전: *"작업 목적과 사용할 도구를 짧게 설명"*
- 작업 완료 후: *"변경된 사항을 리스트 형태로 요약 보고"*
- 모호성 발생 시: 임의로 판단하지 말고 질문하여 의도를 파악합니다.

🌐 범용적 정체성 정의

에이전트가 어떤 프로젝트에서도 일관된 톤앤매너(한국어 사용, 전문적 태도)를 유지하도록 강제하는 기본 레이어입니다.

※ LLM의 기본 시스템 프롬프트보다 높은 우선순위를 가집니다.

🛡️ 행동 경계 및 안전 장치

파괴적인 작업에 대한 사용자 승인 절차를 명시하여, 자동화 과정에서 발생할 수 있는 사고를 미연에 방지합니다.

🗣️ 커뮤니케이션 프로토콜

'생각-실행-보고'의 단계를 구조화하여 인간이 에이전트의 작업 과정을 실시간으로 모니터링하기 쉽게 만듭니다.

※ Human-in-the-loop 구현의 핵심 지침입니다.

Appendix: 예시 Project-specific Local Rule

특정 프로젝트의 성격과 요구사항에 맞춘 지역 규칙의 구체적인 작성 사례를 보여줍니다.

.clinerules markdown

```
# Project: Smart-Inventory-Dashboard
## Tech Stack Context
- Backend: FastAPI (Python 3.11)
- Frontend: React with Tailwind CSS
- DB: PostgreSQL (On-premise internal instance)
## Project Rules
1. Naming: All API routes must use kebab-case.
2. Security: Do NOT use external CDNs for JS/CSS.
3. Error Handling: Use the custom `AppException` class.
4. Auth: Every endpoint requires `internal_token_required` dependency.

## File Boundaries
- Never modify files in `/legacy/v1` folder.
- All new models must be defined in `src/models/*.py`.
```

📌 프로젝트 루트 디렉토리에 위치하며 Global Rule보다 우선 적용됩니다.

🔒 컨텍스트 격리 (Context Isolation)

에이전트에게 현재 작업 중인 프로젝트의 특정 기술 스택과 라이브러리 버전을 명시하여 엉뚱한 코드 생성을 방지합니다.

🛡️ 보안 및 규정 준수

폐쇄망(On-premise) 환경에서의 제약 사항(외부 자원 사용 금지 등)을 명시하여 보안 위배 사항을 사전 차단합니다.

🚧 작업 경계 설정

수정 금지 폴더나 특정 패턴의 파일 위치를 지정하여 에이전트의 활동 범위를 안전하게 제어합니다.

Source: Agentic Workflow Engineering Guide & Project-specific Harness Case Study (2026)

Appendix: 예시 Daily Report Template

에이전트가 일일 보고서를 생성할 때 참조할 수 있는 표준화된 마크다운 템플릿 구조입니다.

📄 템플릿 구조화의 목적

- **일관된 데이터 추출:** 고정된 섹션을 통해 LLM이 작업 내역을 체계적으로 분류하도록 유도
- **히스토리 추적성:** 날짜별로 정형화된 기록을 남겨 추후 검색 및 워킹 노트 연동 최적화
- **자동화 연동:** 생성된 Markdown을 Git 커밋 메시지나 프로젝트 관리 도구(Obsidian 등)로 자동 전송

📌 해당 템플릿은 `.dinerules` 또는 프로젝트 루트의 `TEMPLATE.md`에 저장하여 참조합니다.

DAILY_REPORT_TEMPLATE.md

Daily Report - [YYYY-MM-DD]

1. 오늘의 요약 (Executive Summary)

> 오늘 수행한 작업의 핵심 성과를 1~2문장으로 요약합니다.

2. 완료된 작업 (Completed Tasks)

- [Task ID] 작업 내용 (관련 커밋: #hash)
- [Feature] 신규 기능 구현 및 로컬 테스트 완료
- [Docs] 기술 설계서 업데이트 (docs/arch.md)

3. 진행 중 및 이슈 (In-progress & Issues)

- [Issue] 외부 API 응답 지연 발생 (추적 중)
- [Draft] 리팩토링 모듈 초안 작성 완료

4. 내일의 계획 (Next Steps)

1. 이슈 원인 분석 및 패치 적용
2. 주간 보고서 데이터 취합

Generated by Cline Agent @ [HH:mm:ss]

Appendix: 예시 Git Workflow Instruction

Git 작업을 수행할 때 에이전트가 준수해야 할 단계별 지시문과 절차의 예시입니다.

 skill-git-workflow.md

Git Operations Standard Procedure

1. ****Pre-Task Check****

- 현재 브랜치 확인: ``git branch --show-current``
- 로컬 상태 동기화: ``git fetch origin``
- 작업 시작 전 ``git status``로 깨끗한 상태인지 확인

2. ****Branching Strategy****

- 새로운 기능은 ``feature/JIRA-ID_description`` 형식 사용
- 버그 수정은 ``fix/JIRA-ID_description`` 형식 사용

3. ****Atomic Commit Rule****

- 하나의 커밋에는 하나의 논리적 변경사항만 포함
- 커밋 전 ``git diff``를 통해 불필요한 공백이나 디버그 코드 확인

4. ****Commit Message Template****

- ``type: short description`` (예: ``feat: add email summary logic``)
- 상세 설명이 필요한 경우 한 줄 띄우고 본문 작성

5. ****Review & Push****

- ``git push`` 전 반드시 로컬 빌드/테스트 수행
- Push 후 PR 링크를 사용자에게 보고

 거버넌스 및 제약 사항

강제적 동기화 절차

※ 에이전트가 stale한 코드를 수정하지 않도록 fetch 강제

커밋 메시지 표준화

※ 히스토리 추적성 및 자동 릴리즈 노트 생성 기반

브랜치 명명 규칙

※ 조직의 프로젝트 관리 도구(Jira 등)와 연동성 확보

 Human-in-the-loop 검증

커밋 전 Diff 검토 요청

※ 에이전트가 ``git commit``을 날리기 전 변경 내용을 사용자에게 확인받도록 설정

Push 권한 통제

※ 보안상 중요 프로젝트는 Push 단계를 인간의 승인 후에만 수행

Source: Git Workflow Instruction Example for Cline Agent System